

Phase Correlation for EXOhSPEC Frame-Shift Measurement

Validation Against Live Centroid Tracking and ROI-Size Optimisation

Biswajit Jana
University of Hertfordshire
biswajitj998@gmail.com

August 4, 2026

Abstract

I tested whether phase correlation can independently reproduce the frame-to-frame pixel drift already measured by my live centroid tracker, using the thermal-impulse run GRATING_B_R01_20260622_144720 (72 frames; BASELINE, PULSE, RELAX). My first implementation, following the standard phase-only normalisation recipe, failed: it recovered only $\sim 10\%$ of the true drift amplitude on a large ROI and pure noise on a small one (correlation with the live centroid of 0.19–0.53). I traced this to spectral whitening destroying the signal on my smooth, low-contrast stellar target, and fixed it by switching to standard (non-phase) cross-power-spectrum normalisation with light background-only pre-processing. The corrected method reaches 0.983 (ΔX) and 0.998 (ΔY) correlation with the live centroid, RMSE 0.027–0.029 px. A follow-up ROI-size sweep shows correlation alone is a misleading metric – it stays high even at full-frame – while the recovered drift amplitude collapses from 0.80 px to 0.35 px as the ROI grows to the whole detector. I recommend a 50–200 px ROI centred on the tracked star as the operating point, and flag stellar-SNR robustness as the remaining test before this feeds a closed-loop stabilisation system.

Keywords: phase correlation, image registration, EXOhSPEC, detector drift, centroid tracking, closed-loop stabilisation

1 Introduction

My live EXOhSPEC pipeline tracks one centroided reference star and reports $\Delta X_{\text{centroid}}(t)$, $\Delta Y_{\text{centroid}}(t)$. It’s fast, but it doesn’t tell me whether the whole frame is moving rigidly or whether I’m just watching one local feature. I wanted an independent check, so I ran Fourier-domain phase correlation on the same frames and compared it against the live centroid.

The first version I built didn’t work – it disagreed badly with the live centroid. Diagnosing why is as much the point of this report as the final validated result, since the same failure would silently affect any future registration pipeline built the same way. Section 4 covers the failure and the fix, Section 5 the validated method, Section 6 the ROI-size sweep, Section 7 a direct before/max-drift frame comparison, and Section 10 what this means for closed-loop stabilisation on real stars.

Dataset: GRATING_B_R01_20260622_144720, thermal impulse (8.80 V, 0.25 A, 70 s pulse, 15 min baseline), 72 frames: BASELINE (23), PULSE (2), RELAX (47).

2 Objectives

1. Build a phase-correlation pipeline and compare it against the live centroid.

2. Diagnose and fix any disagreement rather than accepting the first result.
3. Find the ROI size and placement that maximises agreement with the live centroid.
4. Quantify residuals, correlation, and runtime.
5. Assess suitability as an independent cross-check for closed-loop stabilisation on real stars.

3 Background Theory

For reference image $f(x, y)$ and shifted image $g(x, y) = f(x - \Delta x, y - \Delta y)$, the Fourier shift theorem gives

$$G(u, v) = F(u, v) e^{-i2\pi(u\Delta x + v\Delta y)}. \quad (1)$$

3.1 Normalisation Choice

Two normalisations of the cross-power spectrum $F(u, v)G^*(u, v)$ are in common use:

$$R_{\text{phase}}(u, v) = \frac{F(u, v)G^*(u, v)}{|F(u, v)G^*(u, v)|}, \quad (2)$$

$$R_{\text{none}}(u, v) = F(u, v)G^*(u, v). \quad (3)$$

Eq. 2 is true phase correlation: every frequency is rescaled to unit magnitude, so noise and signal contribute equally. It's the standard textbook choice [1, 2] and works well on sharp, high-contrast targets. Eq. 3 keeps the natural amplitude weighting, so real signal dominates over noise. This distinction turned out to be the whole story for this dataset (Section 4). The inverse transform of either gives a correlation surface whose peak is the shift [3].

I used `skimage.registration.phase_cross_correlation` [2, 4], which upsamples around the peak for sub-pixel precision [5]; `upsample_factor=100` gives 0.01 px resolution, well below my ~ 0.1 – 0.8 px drift.

4 Why My First Implementation Failed

My first pipeline used percentile clipping, z-score normalisation, and `normalization="phase"` on (a) one large ROI, (b) five ROIs median-combined, and (c) a small ROI at the tracked star.

Table 1: Correlation with live centroid, first (phase-normalised) implementation.

Variant	corr(ΔX)	corr(ΔY)
Multi-ROI median	0.186	0.529
Local ROI at reference star	0.006	0.015

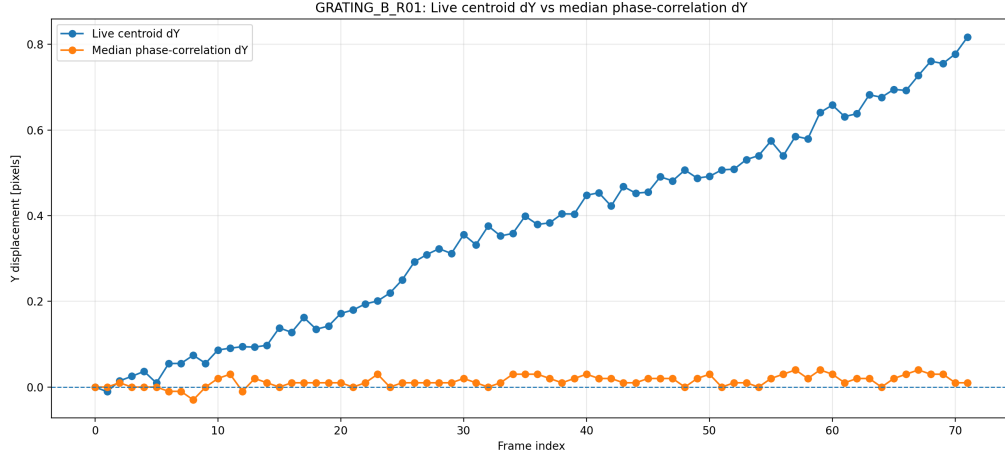


Figure 1: First implementation: live centroid ΔY (real drift up to 0.8 px) vs. multi-ROI phase correlation. The estimate stays pinned near zero.

The built-in error metric didn't help either – it sat at ≈ 1.0 for all 72 frames regardless of quality.

Root cause: my target is a smooth, extended, low-contrast star field, not a sharp edge. Most of its real signal sits in a few low frequencies. Phase-only normalisation flattens every frequency to equal weight, wiping out that signal-to-noise advantage. Combined with percentile clipping and z-scoring (which flattens amplitude further), the correlation peak ends up noise-dominated: diluted toward zero on a large ROI, pure noise on a small one.

Fix (all three applied together):

1. `normalization=None` instead of "phase".
2. Background subtraction only (median), no clipping or z-score.
3. Tukey window ($\alpha = 0.2$) on the ROI before the FFT, to suppress edge artefacts without discarding the flat centre [6] – less aggressive than a Hann taper [7].

5 Validated Results

I ran the corrected pipeline on all 72 frames against a fixed seed frame (frame 0), on a 1000×1000 px ROI centred on the live-centroid reference point ($X_{\text{ref}} = 3959$, $Y_{\text{ref}} = 2984$), `upsample_factor=100`.

Table 2: Corrected method vs. live centroid, 72 frames.

Metric	ΔX	ΔY
Correlation	0.983	0.998
RMSE [px]	0.027	0.029

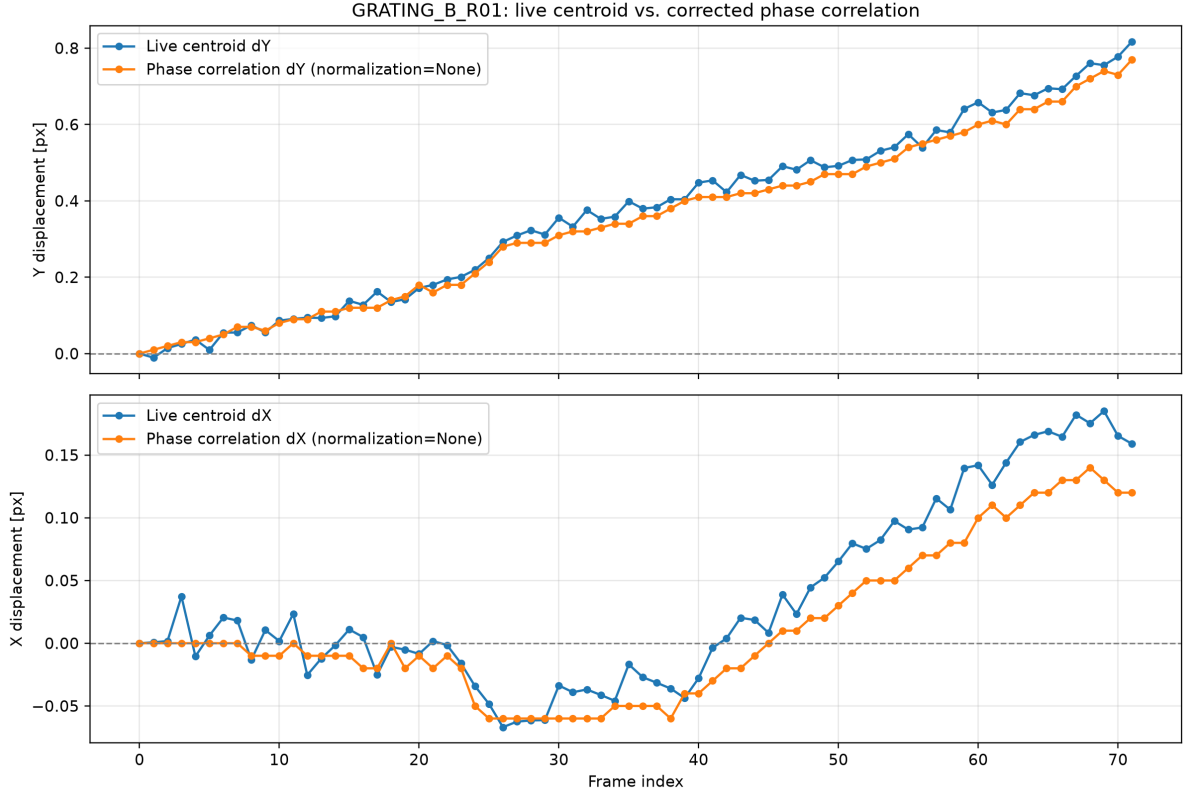


Figure 2: Corrected method: live centroid vs. phase correlation, ΔY (top) and ΔX (bottom), shared frame axis. Tracks closely through BASELINE/PULSE/RELAX, with mild ΔX damping at the largest excursions.

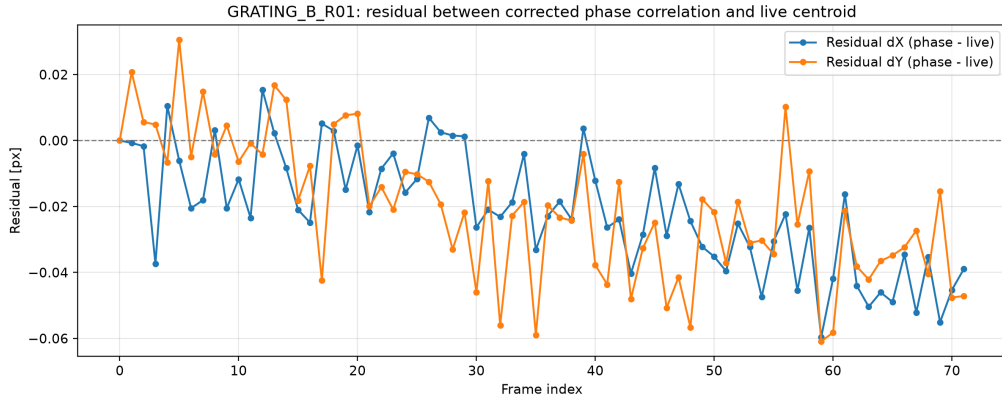


Figure 3: Residual (phase – live). Stays within ± 0.05 px for ΔY ; small systematic growth in ΔX at large cumulative drift.

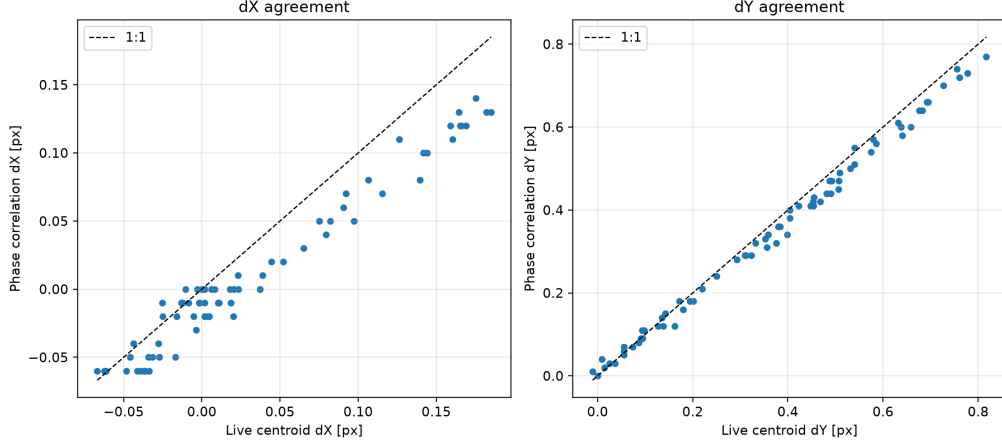


Figure 4: Scatter agreement with the 1:1 line. Both axes cluster tightly on it.

The small ΔX damping (Figure 3) is likely a mild PSF/flux change from the thermal pulse itself, or a small Tukey-taper effect. It doesn't change the overall conclusion.

6 ROI-Size Optimisation

I swept six ROI sizes, 10 px to full-frame (6388×9576), against ten sampled frames.

Table 3: Agreement vs. ROI side length.

ROI side [px]	corr(ΔX)	corr(ΔY)	Measured ΔY range [px]	True range [px]
10	0.896	0.997	0.57	0.778
50	0.995	0.999	0.80	0.778
200	0.997	0.999	0.80	0.778
1000	0.986	0.996	0.73	0.778
3000	0.986	0.997	0.71	0.778
9576 (full frame)	0.808	0.998	0.35	0.778

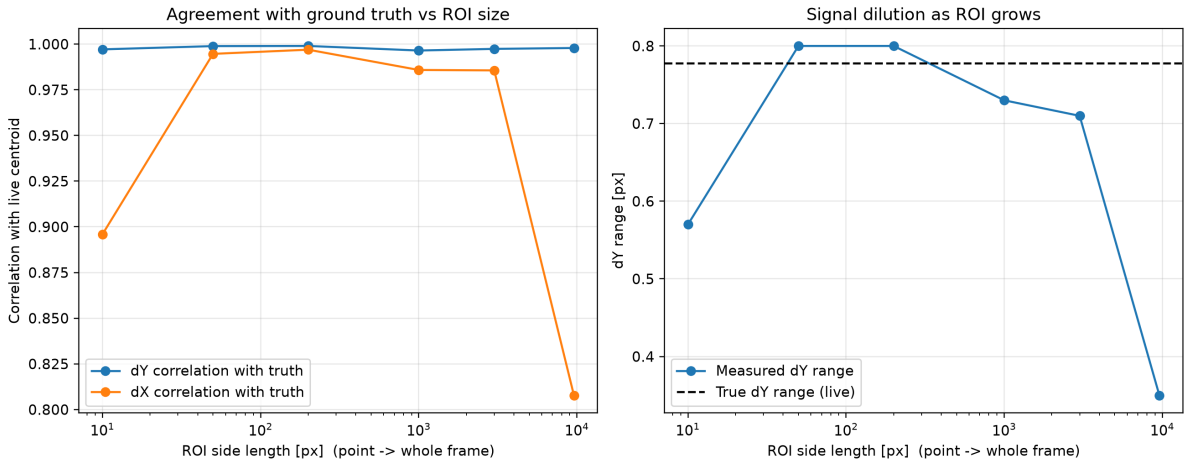


Figure 5: Left: correlation vs. ROI size – stays high (> 0.99) even at full frame. Right: recovered ΔY amplitude vs. the true 0.78 px range – collapses to 0.35 px at full frame. 50–200 px maximises both.

Two failure modes bound the useful range. **Too small (10 px):** ΔX agreement drops (0.90) because the true X drift is small and there isn't enough signal in the box to resolve it against noise. **Too large (full frame):** correlation stays deceptively high, but amplitude collapses – full-frame registration assumes rigid motion, and averaging in the mostly-static background drags the estimate toward zero. **50–200 px** hits both correlation ≥ 0.995 and near-exact amplitude recovery (0.80 vs 0.78 px true).

Recommendation: shrink the operating ROI half-size from 500 px to 100 px, centred on the same reference star as the live centroid.

7 Before vs. Max-Movement Frame

Figure 6 compares the first BASELINE frame against the RELAX frame closest to maximum drift ($\Delta Y \approx 0.74$ px, near the true maximum of 0.77 px at the final frame). At native resolution, zoomed to the tracked ROI, the two frames look essentially identical to the eye – which is exactly why sub-pixel registration is needed here: the drift this report measures is well below what a human blink-comparison can detect directly.

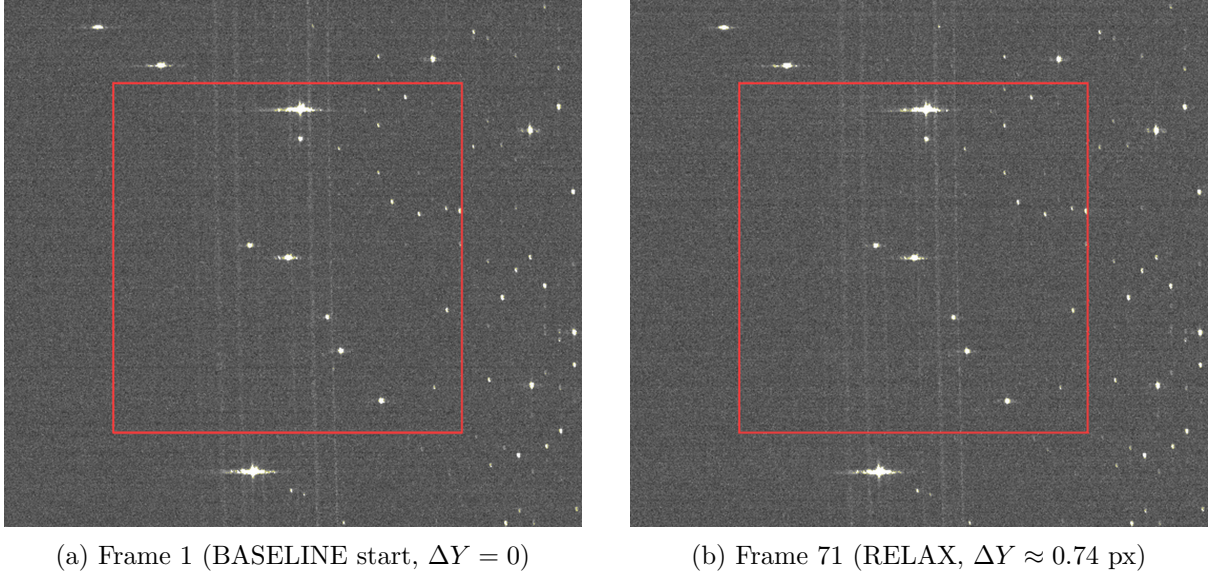


Figure 6: ROI-zoomed comparison, initial vs. near-maximum drift. Visually indistinguishable despite ~ 0.74 px of real motion – the case for phase correlation / centroiding over eyeballing.

8 Full-Frame Visual Context

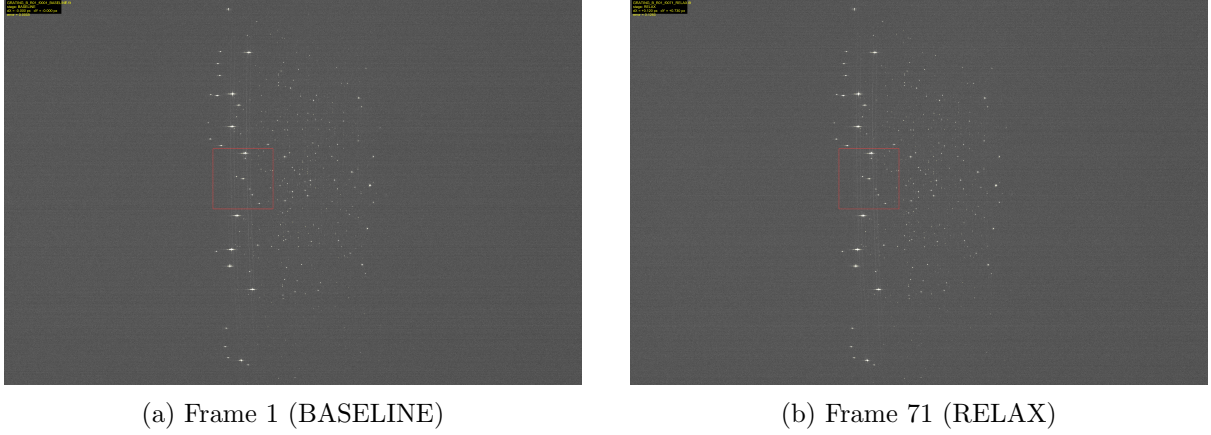


Figure 7: Full detector with the measurement ROI (red box) overlaid, for orientation. The tracked star is deliberately a fainter, unsaturated one, not the bright diffraction-spiked stars just outside the box – saturated cores bias centroiding.

For a talk, I’d pair the full-frame context above with a tight crosshair inset (± 20 – 25 px) so the drift is visible directly rather than only through the numeric overlay.

9 Runtime

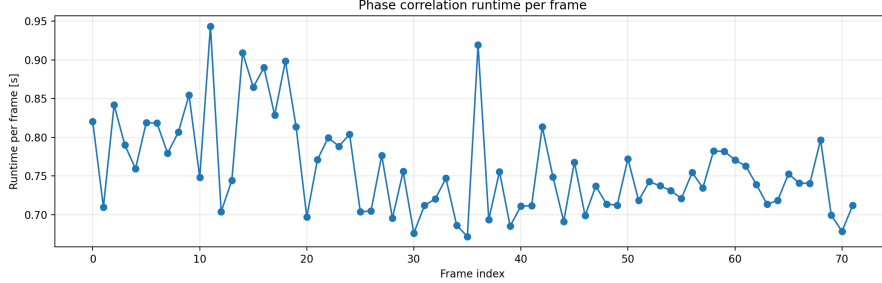


Figure 8: Per-frame runtime, first large-ROI implementation (~ 0.7 – 0.9 s on a 2000×5000 px crop). The corrected method’s much smaller ROI (200–1000 px) should cost proportionally less; needs re-benchmarking on the live system before real-time use.

10 Closed-Loop Implications

My end goal is a closed-loop system stabilising the spectrum on real stars, not a ThAr lamp. That distinction matters: ThAr lines are sharp and high-contrast – easy for almost any method, including the phase-only one that failed here. Real stars are smooth and low-contrast – the harder case, and the one I actually tested. So 0.983–0.998 correlation on a real stellar field is a meaningful proxy for on-sky performance, not just a bench result.

Two things worth flagging for a feedback controller specifically:

1. **Lag:** Figure 2 shares a frame-index axis precisely so lag between live centroid and phase correlation is visible. None seen here, but worth checking on every future run.

2. **SNR (untested):** I swept ROI size, not stellar brightness. A real system will see fainter targets than this run. Next step: repeat Section 5 on lower-SNR frames before trusting this at fainter magnitudes.

11 Interpretation Summary

Result	Interpretation
Phase-normalised, large/multi-ROI	Poor agreement – normalisation/pre-processing killed the signal, not a real physical disagreement.
Phase-normalised, small local ROI	Unusable, noise-dominated.
Corrected, <code>normalization=None</code> , 1000 px ROI	Good agreement – live centroid validated as a reliable local-motion proxy.
ROI-size sweep	Correlation alone is insufficient – full-frame dilutes amplitude while still looking correlated.
50–200 px ROI	Best simultaneous correlation and amplitude recovery – recommended operating point.

12 Limitations

- Assumes rigid translation only – no rotation, scale, or non-rigid deformation modelling [3, 8].
- Still sensitive to saturated pixels, repeated structure, and edge artefacts; mitigated but not eliminated by the fix in Section 4.
- ROI sweep used 10 sampled frames, not all 72 – worth confirming on the full sequence.
- Stellar-SNR dependence untested (Section 10) – the next thing to check.

13 Summary

Phase correlation, correctly configured, agrees strongly with my live centroid (0.983/0.998 correlation, 0.027–0.029 px RMSE). Getting there required catching a specific failure: phase-only normalisation with contrast-normalising pre-processing quietly produces plausible-looking but badly diluted or noisy displacement estimates on smooth, low-contrast stellar targets. My recommended recipe going forward: `normalization=None`, background-subtraction-only pre-processing, Tukey tapering, 50–200 px ROI centred on the tracked star. Before this feeds a closed-loop system: re-benchmark runtime at the smaller ROI, and test robustness across stellar SNR.

A Final Production Code

The validated cell used for Section 5, with the ROI updated to the 100 px recommendation from Section 6.

Listing 1: Final phase-correlation cell (`normalization=None`, tuned ROI).

```

1 from pathlib import Path
2 import time
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 from astropy.io import fits
7 from skimage.registration import phase_cross_correlation
8 from scipy.signal.windows import tukey
9
10 FRAME_DIR = Path(
11     r"C:\Users\labrat\Biswajit"
12     r"\thermal_impulse_8p80V_0p25A_70s_15minbaseline_v6_3"
13     r"\GRATING_B_R01_20260622_144720"

```

```

14     r"\fits"
15 )
16 LIVE_CSV = FRAME_DIR.parent / "GRATING_B_R01_20260622_144720.csv"
17 OUTPUT_DIR = FRAME_DIR.parent / "phase_correlation_outputs"
18 OUTPUT_DIR.mkdir(exist_ok=True)
19
20 fits_files = sorted(list(FRAME_DIR.glob("*.fit")) + list(FRAME_DIR.glob("*.fits"
21     )))
22
23 # Reference point taken from the live centroid system (cx_raw, cy_raw)
24 REF_X, REF_Y = 3959, 2984
25 HALF = 100 # validated sweet spot from the ROI-size sweep (Section 6)
26 CROP = (REF_Y - HALF, REF_Y + HALF, REF_X - HALF, REF_X + HALF)
27
28 _window_cache = {}
29
30 def load_fits_image(path, crop=None):
31     path = Path(path)
32     with fits.open(path, memmap=True, do_not_scale_image_data=True) as hdul:
33         hdu = next(h for h in hdul if getattr(h, "data", None) is not None)
34         data = hdu.data
35         if crop is not None:
36             y0, y1, x0, x1 = crop
37             data = data[y0:y1, x0:x1]
38         return np.asarray(data, dtype=np.float32)
39
40 def preprocess_image(img):
41     """Background-subtract only -- keep amplitude info for cross-correlation."""
42
43     img = np.asarray(img, dtype=np.float32)
44     bad = ~np.isfinite(img)
45     if bad.any():
46         img[bad] = np.nanmedian(img)
47     img = img - np.median(img)
48
49     h, w = img.shape
50     key = (h, w)
51     if key not in _window_cache:
52         wy = tukey(h, alpha=0.2)
53         wx = tukey(w, alpha=0.2)
54         _window_cache[key] = np.outer(wy, wx).astype(np.float32)
55     return img * _window_cache[key]
56
57 def cross_correlate_pair(reference_img, moving_img, upsample_factor=100):
58     t0 = time.perf_counter()
59     shift_yx, error, phasediff = phase_cross_correlation(
60         reference_img,
61         moving_img,
62         upsample_factor=upsample_factor,
63         normalization=None, # the fix: NOT "phase"
64     )
65     runtime = time.perf_counter() - t0
66     return {
67         "dX_phase": -float(shift_yx[1]),
68         "dY_phase": -float(shift_yx[0]),
69         "phase_error": float(error),
70         "phase_diff": float(phasediff),
71         "runtime_sec": runtime,
72     }
73
74 # Run: all frames vs a fixed seed frame (avoids accumulated error)
75 rows = []
76 seed = preprocess_image(load_fits_image(fits_files[0], CROP))

```

```

75
76 for i, path in enumerate(fits_files):
77     moving = preprocess_image(load_fits_image(path, CROP))
78     result = cross_correlate_pair(seed, moving, upsample_factor=100)
79
80     name_upper = path.name.upper()
81     stage = ("BASELINE" if "BASELINE" in name_upper else
82             "PULSE" if ("PULSE" in name_upper or "HEAT" in name_upper) else
83             "RELAX" if ("RELAX" in name_upper or "RECOVERY" in name_upper)
84             else
85             "UNKNOWN")
86
87     rows.append({"frame_index": i, "file_name": path.name, "stage": stage, **
88                result})
89
90 cc_df = pd.DataFrame(rows)
91 cc_df.to_csv(OUTPUT_DIR / "cross_correlation_FINAL.csv", index=False)
92
93 # Validate against the live centroid ground truth
94 live_df = pd.read_csv(LIVE_CSV)
95 live_df["file_name"] = live_df["fits_filename"].astype(str).apply(lambda p:
96                             Path(p).name)
97 merged = cc_df.merge(live_df[["file_name", "dX", "dY"]], on="file_name", how="
98                     inner")
99
100 print("corr dX:", merged["dX"].corr(merged["dX_phase"]))
101 print("corr dY:", merged["dY"].corr(merged["dY_phase"]))

```

References

- [1] OpenCV. OpenCV PhaseCorrelate Documentation. <https://docs.opencv.org/>, 2026. Documentation for phase correlation, Fourier shift theorem, Hanning windowing, and response metric. Accessed July 2026.
- [2] scikit-image contributors. scikit-image Registration API: phase_cross_correlation. <https://scikit-image.org/docs/stable/api/skimage.registration.html>, 2026. Documentation for Fourier-domain phase cross-correlation, subpixel upsampling, normalisation, error, and phase difference outputs. Accessed July 2026.
- [3] B. Srinivasa Reddy and B. N. Chatterji. An fft-based technique for translation, rotation, and scale-invariant image registration. *IEEE Transactions on Image Processing*, 5(8):1266–1271, 1996. doi: 10.1109/83.506761.
- [4] Stéfan van der Walt, Johannes L. Schönberger, Juan Nunez-Iglesias, François Boulogne, Joshua D. Warner, Neil Yager, Emmanuelle Gouillart, and Tony Yu. scikit-image: image processing in python. *PeerJ*, 2:e453, 2014. doi: 10.7717/peerj.453.
- [5] Manuel Guizar-Sicairos, Samuel T. Thurman, and James R. Fienup. Efficient subpixel image registration algorithms. *Optics Letters*, 33(2):156–158, 2008. doi: 10.1364/OL.33.000156.
- [6] SciPy contributors. SciPy Documentation: scipy.signal.windows.tukey. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.windows.tukey.html>, 2026. Documentation for the Tukey (tapered cosine) window, used in this work in place of a Hann window to reduce FFT edge artefacts while retaining more of the flat central region of the ROI. Accessed July 2026.

- [7] SciPy contributors. SciPy Documentation: `scipy.signal.windows.hann`. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.windows.hann.html>, 2026. Documentation for the Hann window. Accessed July 2026.
- [8] scikit-image contributors. scikit-image Example: Using Polar and Log-Polar Transformations for Registration. https://scikit-image.org/docs/0.25.x/auto_examples/registration/plot_register_rotation.html, 2026. Documentation example discussing translation registration and the limitation under rotation or scale changes. Accessed July 2026.